Quantum CompuAting and Quantum Machine Learning

Morten Hjorth-Jensen[1]

Department of Physics, University of Oslo, Norway[1]

January 21, 2026

Overview of first week, Basic Notions of Quantum Mechanics

Linear Algebra reminders, Hilbert Spaces, operators on Hilbert spaces, states, qubits and other definitions

1. Mathematical notation, Hilbert spaces and operators
2. Description of quantum systems and one-qubit systems
3. States in Hilbert space, pure and mixed states
4. Operators and simple gates
5. Video of lecture at https://youtu.be/J5lK-fTcTYY
6. Whiteboard notes at https://github.com/CompPhysics/QuantumComputingMachineLearning/tree/gh-pages/doc/HandWrittenNotes/2026/Lectureweek1.pdf

Reading recommendation

1. Scherer, Mathematics of Quantum Computations, chapter 2 at https://link.springer.com/book/10.1007/978-3-030-12358-1
2. Hidary, Quantum Computing: An Applied Approach, chapter 12 at https://link.springer.com/book/10.1007/978-3-030-23922-0
3. You may also find chapters 1-2 of Olivares useful, see https://link.springer.com/book/10.1007/978-3-031-83361-8

These chapters from different textbooks give you an overview of some of the central (often linear algebra) mathematical elements needed for quantum computing. These lecture notes aim at giving you a basic overview of central mathematical elements. For more details we refer you to the above texts and the companion lecture notes.

Practicalities

1. Weekly lectures with weekly exercise sessions and assignments. The assignments are meant as background for the two projects
2. We plan to work on two projects which will define the content of the course

More on projects and final grade

1. Two projects which count 50% each for the final grade
2. Deadline first project March 20, see https://github.com/CompPhysics/QuantumComputingMachineLearning/tree/gh-pages/doc/Projects/2026/Project1
3. Deadline second project June 1
4. All info at the GitHub address https://github.com/CompPhysics/QuantumComputingMachineLearning

Topics for the projects

1. First project: Quantum computing and simulation of quantum mechanical systems using the VQE algorithm
2. Second project: Continuation of the first topic with more realistic systems and using adaptive VQE
3. Second project: Applications and implementations of quantum machine learning algorithms
4. Second project: Studies of entanglement and physical realization of quantum gates and circuits

More topics for the second projects

1. Second project: Implementation of quantum simulators on actual (real) quantum computers
2. Second project: Simulation of linear algebra systems on quantum computers, solving differential equations
3. Second project: Implementation and studies of the Quantum Approximate Optimization Algorithm (QAOA)
4. Second project: Implement quantum neural networks for PINNs
5. Second project: Implementing Shor's algorithm or algorithms or other topics ....

Project two examples at https://github.com/CompPhysics/QuantumComputingMachineLearning/tree/gh-pages/doc/Projects/ProjectExamples

Schedule

1. Lectures: Wednesday 1015-12, first lecture January 21. Last lecture May 13. Room FØ434
2. Exercise sessions: Wednesday 815-10, first session January 28. Last session May 20. Room FØ434

Possible textbooks, Quantum Computing

1. Wolfgang Scherer, Mathematics of Quantum Computing, see https://link.springer.com/book/10.1007/978-3-030-12358-1
2. Hidary, Quantum Computing: An Applied Approach, see URL:https://link.springer.com/book/10.1007/978-3-030-23922-0"
3. Stefano Olivares, A Student's Guide to Quantum Computing, see https://link.springer.com/book/10.1007/978-3-031-83361-8
4. Robert Hundt, Quantum Computing for Programmers, https://www.cambridge.org/core/books/quantum-computing-for-programmers/BA1C887BE4AC0D0D5653E71FFBEF61C6
5. Robert Loredo, Learn Quantum Computing with Python and IBM Quantum Experience, see https://github.com/PacktPublishing/Learn-Quantum-Computing-with-Python-and-IBM-Quantum-Exp

Possible textbooks, Quantum ML

1. Maria Schuld and Francesco Petruccione, Machine Learning with Quantum Computers, see https://link.springer.com/book/10.1007/978-3-030-83098-4
2. Claudio Conti, Quantum Machine Learning (Springer), see UR:"https://link.springer.com/book/10.1007/978-3-031-44226-1"

Classic texts and lecture notes

1. Michael A. Nielsen and Isaac L. Chuang, Quantum Computation and Quantum Information, see https://www.cambridge.org/highereducation/books/quantum-computation-and-quantum-information/01E10196D0A682A6AEFFEA52D53BE9AE#overview
2. Jack Hidary, Quantum Computing : An Applied Approach, see https://link.springer.com/book/10.1007/978-3-030-23922-0
3. Lectures by John Preskill at https://www.preskill.caltech.edu/ph229/

Online material of possible interest

1. See IBM's Quantum Computer Programming: Hands-On Workshop at https://quantgates.com/learn-quantum
2. IBM quantum challenge at https://github.com/qiskit-community/ibm-quantum-challenge-2024?tab=readme-ov-file
3. IBM learning at https://learning.quantum.ibm.com/course/
4. Coding with Qiskit https://www.youtube.com/watch?v=93-zLTppFZw and https://github.com/derek-wang-ibm/coding-with-qiskit/tree/main
5. Coding with QuTip, see https://qutip.org/
6. Qbraid at https://www.qbraid.com
7. PennyLane at https://pennylane.ai/(tailoredtomachinelearning)

What is quantum computing?

It is computation (and more generally, information processing) based on the principles of quantum mechanics, rather than classical physics. Quantum mechanics is a description of nature at its most fundamental level.

Quantum mechanics, Formulated in the early 20th century, and its specializations (quantum field theory, quantum chromodynamics, many-body physics etc.), have been spectacularly successful in explaining microscopic physical phenomena.

The exponentiality of QM

After nearly a century of study, the best (classical) methods for predicting the behavior of general quantum systems require exponential resources.

1. The state of N particles requires at least 2^N numbers to describe.
2. For $N \sim 300$ (the number of particles in a single uranium atom and looking only at spin degrees freedom),

$$2^{300} \gg\approx 10^{82} \quad (\# \text{ of atoms in the observable universe}).$$

Nature seems to be doing extravagant amounts of computation *behind the scenes*!

The exponentiality of QM I

How do physicists actually do (quantum) physics?

1. Answer 1: 100+ years of extremely clever approximations and shortcuts for calculations.
   - Examples: Bethe ansatz, Feynman diagrams, perturbation theory, mean field approximations, ...
2. Answer 2: Computer simulations
   - Examples: Density functional theory, Quantum Markov Chain Monte Carlo, Full Configuration interaction theory, Coupled-Cluster, DMRG, ...
3. Answer 3: studying systems that allow for Answers 1 and 2 above
   - Systems that are mostly classical
   - Very often we simualte systems with non-strongly interacting particles (1), but can do strongly interacting (Monte Carlo methods)
   - Most systems studied assume that a single-particle basis is a good approximation.

But simulating general quantum physics problems is still a hard

The exponentiality of QM II

In 1981, in his talk titled *"Simulating Physics with Computers"*, Richard Feynman asked the following question: **Can probabilistic computers simulate quantum mechanics?**.

His conclusion, after a lengthy exploration was that quantum mechanics [doesn't] seem to be imitable by a local classical computer. Instead we need a computer built of quantum mechanical elements which obey quantum mechanical laws. Feynman, along with David Deutsch, Paul Benioff and Yuri Manin, are credited with the idea of computing based on quantum mechanical principles.

The early days

1. Feynman's motivation for quantum computers is the most obvious one: simulating quantum systems.
2. To physicists in the '80s, this idea might have seemed obvious. To a computer scientist in the '80s, QCs would have seemed like a wild crackpot idea.
3. Also in the '80s, David Deutsch contemplated other applications of quantum computers. He gave an example of a (classical) problem where the "parallel universes" of QM seem to speed up (by a constant factor).
4. 1992: Dan Simon came up with an example of a problem that could be solved exponentially faster on a QC.

Shor's breakthrough

1. 1993: Peter Shor realized that Simon's idea could be extended to efficiently solve the following problem:
2. **Integer Factorization:** Given integer $N > 0$, find integer $M > 1$ such that M divides N.
3. The security of the RSA cryptosystem (the most widely deployed public-key encryption on the internet) crucially relies on the assumption that factoring integers is hard!

Crossroads

Since Shor's algorithm, physicists and computer scientists have been faced with three options:

1. Quantum mechanics is wrong.
2. There is a fast classical algorithm for factoring.
3. Quantum computers are more powerful than classical computers.

At least one of these must be true! Which do you think is most likely to be true?

Which is wrong?

1. Option 1: QM is wrong
   - Perhaps the most successful theory of nature to date.
   - In all its domains of applicability, we have never found experimental disagreement (for example test of the Pauli principle).
2. Option 2: Polynomial-time classical factoring algorithm.
   - RSA has been out for nearly 50 years. There is enormous economic pressure to discover weaknesses, if it exists.
   - The **Gauss argument**: if the Prince of Mathematics couldn't find it, then it probably doesn't exist.

More

1. Option 3: Quantum computers are more powerful than classical computers.
   - This would refute the Extended Church–Turing Thesis.
   - Church–Turing Thesis: A Turing machine can simulate any effective computation process.
   - Extended Church-Turing (ECT) Thesis: A probabilistic Turing machine can efficiently simulate any effective computation process.
   - Option 3 would violate the ECT (assuming factoring is hard for classical computers)!

The options

Since Shor's algorithm, physicists and computer scientists have been faced with three options:

1. Quantum mechanics is wrong.
2. There is a fast classical algorithm for factoring.
3. Quantum computers are more powerful than classical computers.

At least one of these must be true!

Naive yet popular statement

A naive, yet popular, statement says that quantum computers can provide an exponential speedup over classical computers because their internal states live in an exponentially large space of dimension 2^N for an N-qubit quantum computer. The number of dimensions grows so fast with N that classical supercomputers cannot even hold this state in memory as soon as $N > 50$; hence, classical computing is supposedly doomed to address these states.

More to the picture than meets the eye

Yet, despite this supposed impossibility, a rather large number of such exponentially large states have been calculated, sometimes with machine precision, using classical algorithms.
The solution of this small paradox is the same as in other successes of physics: apparently very complex phenomena have internal mathematical structures that, when revealed, allow one to make precise predictions.

Present day

Quantum computing is an extremely active and exciting field:

1. Quantum engineering: quantum computers are being built.
2. Theoretical developments: new algorithms, new cryptographic protocols, new insights into how quantum computing compares to classical computing.
3. Deep connections to other sciences: condensed matter physics, quantum gravity, materials science, chemistry, computer science, pure mathematics. Fundamental aspects of quantum mechanics.

Emerging quantum computers

1. Until 2017 or so, most quantum computing devices had ~ 10 qubits.
2. Suddenly, there was a flood of resources devoted to quantum engineering efforts, and now we are seeing programmable quantum computers with $50+$ qubits.

Major problem: devices are insanely noisy! We are in the "NISQ" era:

1. Noisy Intermediate-Scale Quantum
2. Noisy devices with ~ 100 qubits.
3. Not capable of running Shor's algorithm, but should still be capable of solving interesting, hard problems.

Quantum supremacy

1. In Fall 2019, Google announced that they had achieved "Quantum Supremacy" using their 53-qubit Sycamore quantum processor.
2. **Quantum supremacy:** a moment when there is a convincing real-world demonstration of a quantum computing task that cannot feasibly be performed by a classical computer.

Summary of hardware efforts

1. We are in the NISQ era, but we have compelling evidence that we can already perform super-classical tasks using these machines.
2. There is a **qubit race** going on, but qubit count is not everything!
3. Scaling up (i.e. more, higher-quality qubits) is a tough quantum engineering challenge, but no fundamental obstacles to building a QC with 10^6 noiseless qubits.
4. QUNorth in Denmark and LUMIQ as a European project
5. Still, large-scale fault-tolerant QCs look like they're many years away.

Quantum algorithms

1. Exponential speedups for structured, algebraic problems
2. Factoring (Shor's algorithm)
3. Polynomial speedups for unstructured search problems
4. Grover search
5. Exponential speedups for quantum simulation
   - Simulating quantum physics and chemistry (Feynman's dream and mine as well). Probably the most important application of quantum computers so far!

Quantum algorithms

Algorithms to be run on near-term QCs:

1. **Variational eigensolvers** and **Variational Quantum Circuits**
2. **Classical–quantum hybrid algorithms**
3. **Quantum Machine Learning**
4. Linear system solvers / SDPs / Convex Optimization
5. **Quantum neural nets**
6. Solving differential equations
7. more

Connections with fundamental physics: Perhaps key to a theory of Quantum Gravity could be quantum entanglement, and quantum error correcting codes.

The basic concepts

Quantum technologies leverage principles of quantum mechanics to perform computations beyond classical capabilities.

Key Concepts:

1. **Superposition:** Qubits can exist in a combination of states.
2. **Entanglement:** Correlation between qubits regardless of distance.
3. **Quantum Interference:** Probability amplitudes interfere. Is a core tool in quantum algorithms. It is used to amplify the probability of finding the correct answer while suppressing incorrect ones.

Quantum Speedups

Why Quantum?

1. **Quantum Parallelism:** Process multiple states simultaneously.
2. **Quantum Entanglement:** Correlated states for richer information.
3. **Quantum Interference:** Constructive and destructive interference to enhance solutions, is a core tool in quantum algorithms.

Grover's Algorithm:

$$\text{Quantum Search Complexity: } O(\sqrt{N}) \text{ vs. } O(N)$$

Advantage: Speedups in high-dimensional optimization and linear algebra problems.

Challenges and Limitations

Quantum Hardware Limitations:

1. Noisy Intermediate-Scale Quantum (NISQ) devices.
2. Decoherence and limited qubit coherence times.

Data Encoding:

1. Efficient embedding of classical data into quantum states.

Scalability:

1. Difficult to scale circuits to large datasets.

1. Quantum Communication

Quantum Teleportation:

1. Entanglement enables the transmission of quantum states using classical communication.
2. No need to send the physical quantum particle.

Advantage:

1. Instantaneous state transfer within quantum mechanics constraints.
2. Quantum networks rely on entanglement for secure communication.

2. Quantum Cryptography

Quantum Key Distribution:

1. Entanglement ensures secure communication.
2. Eavesdropping disturbs quantum states, revealing interception attempts.

1. Any measurement by a third party collapses the wavefunction.
2. Ensures security based on quantum mechanics, not computational hardness.

Advantage: Unconditional security guaranteed by the laws of physics.

3. Quantum Computing

Speedup in Quantum Algorithms:

1. Entanglement provides exponential state space.
2. Quantum parallelism arises from entangled qubits.

4. Quantum Metrology

Quantum Metrology:

1. Uses entangled states for ultra-precise measurements.
2. Overcomes the classical shot-noise limit.

1. Quantum entanglement improves sensitivity beyond classical limits.

Challenges of Quantum Entanglement

Decoherence:

1. Entangled states are fragile.
2. Interaction with the environment collapses the wavefunction.

Scalability:

1. Difficult to entangle large numbers of qubits.
2. Error correction requires complex protocols.

Measurement Problem:

1. Measurement destroys entanglement.
2. Trade-off between information gain and entanglement preservation.

What is Quantum Machine Learning?

Quantum Machine Learning (QML) integrates quantum computing with machine learning algorithms to exploit quantum advantages.

Motivation:

1. High-dimensional Hilbert spaces for better feature representation.
2. Quantum parallelism for faster computation.
3. Quantum entanglement for richer data encoding.

Di Vincenzo criteria

Quantum computing requirements

1. A scalable physical system with well-characterized qubit
2. The ability to initialize the state of the qubits to a simple fiducial state
3. Long relevant Quantum coherence times longer than the gate operation time
4. A **universal** set of quantum gates
5. A qubit-specific measurement capability

Quantum platforms

1. Superconducting qubits use Josephson junction circuits operated at millikelvin temperatures
2. Trapped-ion computers confine ions (for example $^{171}Yb^{+}$, $^{40}Ca^{+}$) in electromagnetic traps, encoding qubits in internal electronic states. Gates are implemented with laser or microwave fields coupling to vibrational modes.
3. Photonic quantum computing employs photons (in dual-rail or continuous-variable encodings) as qubits
4. Spin qubits—realized in silicon quantum dots, donor atoms, or color centers—encode information in electron or nuclear spin states.
5. Electrons on helium/neon and other

Quantum computing in a nutshell

A quantum computer is a well-controlled out-of-equilibrium quantum many-body system that one intends to use to perform a calculation. Such a system can be described at several levels: from the actual underlying physics (usually described in terms of its Hamiltonian, that is with time and energies) up to an abstract representation used to describe quantum algorithms, the so-called **gate-based quantum computer**.

An exponentially large internal state

The abstract **gate-based** quantum computer is defined as follows. We have a set of N two-level systems, called quantum bits or qubits (for instance, the spin of an electron), that can be in the states $|0\rangle$ and $|1\rangle$. The most general state of the quantum computer has the form

$$|\Psi\rangle = \sum_{i_1 i_2 \cdots i_N} \Psi_{i_1 i_2 \cdots i_N} |i_1 i_2 \cdots i_N\rangle.$$

Sums

Here the sum runs over all qubit values $i_a \in \{0, 1\}$, and $|i_1 i_2 \cdots i_N\rangle$ is a shorthand for the tensor product $|i_1 i_2 \cdots i_N\rangle = |i_1\rangle \otimes |i_2\rangle \otimes \cdots \otimes |i_N\rangle$. The tensor $\Psi_{i_1 i_2 \cdots i_N}$ can be thought of as a large vector containing 2^N complex values. The potential capabilities of quantum computers stem from the fact that this vector is exponentially large and, once $N \gtrsim 50$, cannot be stored in a classical computer.

Quantum circuits

When one operates a quantum computer, one initializes it with an initial state $|\Psi\rangle^{(0)}$ (usually $|\Psi\rangle^{(0)} = |000\cdots0\rangle$), and the state of the system evolves according to the Schrödinger equation, which transforms $|\Psi\rangle^{(n)}$ into $|\Psi\rangle^{(n+1)} = \hat{U}^{(n)}|\Psi\rangle^{(n)}$, with the evolution operator $\hat{U}^{(n)}$ given by

$$\hat{U}^{(n)} = Te^{-i\int_{t_n}^{t_{n+1}} dt\hat{H}(t)}.$$

Here where T is the time-ordering operator and $\hat{H}(t)$ is the Hamiltonian of the system.

Unitary evolution operator in quantum computing

In physics, the system is described by $\hat{H}(t)$. In quantum computing, one actually starts with the evolution operators $\hat{U}^{(n)}$, assuming that someone else has worked out how to engineer appropriate Hamiltonians. The different evolution operators that one will use are called **gates** and fall into two categories, depending on whether they act on a single qubit or on two qubits.

Single-qubit gate

A single-qubit gate is defined on one qubit as $\hat{U}|i\rangle = \sum_j U_{ji}|j\rangle$, where the 2×2 matrix U_{ij} is unitary. In terms of the wavefunction $\Psi^{(n)}$, such a single-qubit gate on qubit a translates into a matrix that acts as

$$\Psi^{(n+1)}_{i_1 i_2 \cdots i_N} = \sum_{i'_a} U^{(n)}_{i_a i'_a} \Psi^{(n)}_{i_1 \cdots i_{a-1} i'_a i_{a+1} \cdots i_N}.$$

Two-qubit gate

Likewise, a two-qubit gate acting on qubits a and b is described by a 4×4 matrix and transforms the wavefunction into

$$\Psi^{(n+1)}_{i_1 i_2 \cdots i_N} = \sum_{i'_a, i'_b} U^{(n)}_{i_a i_b, i'_a i'_b} \Psi^{(n)}_{i_1 \cdots i_{a-1} i'_a i_{a+1} \cdots i_{b-1} i'_b i_{b+1} \cdots i_N}.$$

Depending on the quantum hardware, some gates are easier to implement than others.

The Pauli gates

Typical examples include the one-qubit gates (the first three are the Pauli matrices)

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \tag{1}$$

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \tag{2}$$

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \tag{3}$$

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \tag{4}$$

$$T = \begin{pmatrix} e^{i\pi/8} & 0 \\ 0 & e^{-i\pi/8} \end{pmatrix}. \tag{5}$$

And the controlled NOT (or CNOT) two-qubit gate

$$C_X = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

These gates are transformed into so-called quantum circuits (see whiteboard notes) and they are read a little like music, with one line per qubit.

Measurement

The last ingredient of the quantum computer gate model is measurement. When a qubit a is measured, it returns the value α ($\alpha = 0$ or 1) with probability

$$P_\alpha = \sum_{i_1 \cdots i_{a-1} i_{a+1} \cdots i_N} \left| \psi^{(n)}_{i_1 \cdots i_{a-1} \alpha i_{a+1} \cdots i_N} \right|^2 .$$

After measurement

The new wavefunction becomes

$$\Psi^{(n)}_{i_1 \cdots i_N} \to \delta_{i_a,\alpha} \frac{1}{\sqrt{P_\alpha}} \Psi^{(n)}_{i_1 \cdots i_{a-1} \alpha i_{a+1} \cdots i_N}.$$

And that's essentially it. The above set of equations entirely describes what a quantum computer is supposed to do. The entire field of quantum algorithms consists of using these rules to perform useful computations.

Quantum computing in a nutshell

In a nutshell, a quantum computer allows one to perform a subset of linear algebra, namely matrix-vector multiplications, with very special matrices (the **gates** that act only on certain indices and belong to a fixed set of unitary matrices) on exponentially large vectors (the wavefunction $\Psi^{(n)}_{i_1 \cdots i_N}$). The appeal of quantum computers clearly comes from the exponentially large size of those wavefunctions.

Downside

However, a very strong downside is that at the end of the calculation one does not hold the corresponding exponentially large vector of 2^N values, but only a much smaller set: N bits of (probabilistic) information. We get a single sample of the distribution $|\Psi^{(n)}_{i_1 \cdots i_N}|^2$. This is one of the two Achilles' heels of quantum computing, which we dub the I/O bottleneck (well, more the O bottleneck for this aspect).

Inaccesible parts of the space

It is very important to realize that the largest part of the Hilbert space of dimension 2^N will remain forever inaccessible to quantum computers (and classical methods). This can be understood using a simple counting argument. Suppose that the quantum circuit consists of D layers of gates (D being the depth of the circuit). We also suppose that each layer is packed with as many gates as possible (meaning that all the qubits are acted upon). Lastly, each gate is parametrized by a few angles. Then the total dimension of the subspace that can be spanned by these circuits is $O(DN)$, which is obviously much smaller than 2^N.

Some numbers

Now let us put some realistic numbers. Suppose that we work with $N = 100$ qubits. The total dimension of the Hilbert space is $2^{100} \approx 10^{30}$. Typical depths that can be considered with existing hardware are of the order of $D \approx 100$, but let us suppose that this number is scaled up to $D = 10^6$. The explorable subspace would still have 20 orders of magnitude fewer degrees of freedom than the full Hilbert space. So the question really is: does this subspace belong to the **relevant** part of the Hilbert space?

Degrees of freedom

And, conversely, is the **relevant** part of the Hilbert space amenable to classical simulations? The word relevant is defined very loosely here, but there are several scientific articles that start to give it a more precise meaning. For instance, the problem of the **barren plateaus** in the variational quantum eigensolver (VQE) algorithm has been traced back to the fact that most of the states in the $O(ND)$-dimensional subspace manifold are essentially chaotic, hence irrelevant.

Notations and definitions

Vectors, matrices and higher-order tensors are always boldfaced, with vectors given by lower case letter letters and matrices and higher-order tensors given by upper case letters.
Unless otherwise stated, the elements v_i of a vector $\boldsymbol{v}$ are assumed to be real. That is a vector of length n is defined as $\boldsymbol{x} \in \mathbb{R}^n$ and if we have a complex vector we have $\boldsymbol{x} \in \mathbb{C}^n$.
For a matrix of dimension $n \times n$ we have $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ and the first matrix element starts with row element (row-wise ordering) zero and column element zero.

Some mathematical notations

1. For all/any $\forall$
2. Implies $\Longrightarrow$
3. Equivalent $\equiv$
4. Real variable $\mathbb{R}$
5. Integer variable $\mathbb{I}$
6. Complex variable $\mathbb{C}$

Vectors

We start by defining a vector $\boldsymbol{x}$ with n components, with x_0 as our first element, as

$$\boldsymbol{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \dots \\ \dots \\ x_{n-1} \end{bmatrix}.$$

and its transpose

$$\boldsymbol{x}^T = \begin{bmatrix} x_0 & x_1 & x_2 & \dots & \dots & x_{n-1} \end{bmatrix},$$

In case we have a complex vector we define the hermitian conjugate

$$\boldsymbol{x}^\dagger = \begin{bmatrix} x_0^* & x_1^* & x_2^* & \dots & \dots & x_{n-1}^* \end{bmatrix},$$

Inner products of vectors

With a given vector $\boldsymbol{x}$, we define the inner product as

$$\boldsymbol{x}^T\boldsymbol{x} = \sum_{i=0}^{n-1} x_i x_i = x_0^2 + x_1^2 + \cdots + x_{n-1}^2,$$

or in case of a complex vector

$$\boldsymbol{x}^\dagger\boldsymbol{x} = \sum_{i=0}^{n-1} x_i^* x_i = |x_0|^2 + |x_1|^2 + \cdots + |x_{n-1}|^2,$$

Hermitian conjugate

The hermitian conjugate of a matrix is obtained by taking the complex conjugate of each element and then taking the transpose of the resulting matrix. Often we will just say the transpose or just the conjugate, it should be clear from the context that we will mainly deal with hermitian quantities and our matrices will in most cases be square matrices.
Unitarity, as we will see below, plays also a central role in this course.

Outer products

In addition to inner products between vectors/states, the outer product plays a central role in many applications. It is defined as

$$\boldsymbol{xy}^T = \begin{bmatrix} x_0y_0 & x_0y_1 & x_0y_2 & \dots & \dots & x_0y_{n-2} & x_0y_{n-1} \\ x_1y_0 & x_1y_1 & x_1y_2 & \dots & \dots & x_1y_{n-2} & x_1y_{n-1} \\ x_2y_0 & x_2y_1 & x_2y_2 & \dots & \dots & x_2y_{n-2} & x_2y_{n-1} \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ x_{n-2}y_0 & x_{n-2}y_1 & x_{n-2}y_2 & \dots & \dots & x_{n-2}y_{n-2} & x_{n-2}y_{n-1} \\ x_{n-1}y_0 & x_{n-1}y_1 & x_{n-1}y_2 & \dots & \dots & x_{n-1}y_{n-2} & x_{n-1}y_{n-1} \end{bmatrix}$$

The latter defines also our basic matrix layout.

Basic Matrix Features

A general $n \times n$ matrix is given by

$$\boldsymbol{A} = \begin{bmatrix} a_{00} & a_{01} & a_{02} & \dots & \dots & a_{0n-2} & a_{0n-1} \\ a_{10} & a_{11} & a_{12} & \dots & \dots & a_{1n-2} & a_{1n-1} \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ a_{n-20} & a_{n-21} & a_{n-22} & \dots & \dots & a_{n-2n-2} & a_{n-2n-1} \\ a_{n-10} & a_{n-11} & a_{n-12} & \dots & \dots & a_{n-1n-2} & a_{n-1n-1} \end{bmatrix},$$

or in terms of its column vectors $\boldsymbol{a}_i$ as

$$\boldsymbol{A} = \begin{bmatrix} \boldsymbol{a}_0 & \boldsymbol{a}_1 & \boldsymbol{a}_2 & \dots & \dots & \boldsymbol{a}_{n-2} & \boldsymbol{a}_{n-1} \end{bmatrix}.$$

We can think of a matrix as a diagram of in general n rowns and m columns. In the example here we have a square matrix.

The inverse of a matrix

The inverse of a square matrix (if it exists) is defined by

$$\boldsymbol{A}^{-1} \cdot \boldsymbol{A} = I,$$

where $\boldsymbol{I}$ is the unit matrix.

Selected Matrix Features

Relations	Name	matrix elements
$\boldsymbol{A} = \boldsymbol{A}^T$	symmetric	$a_{ij} = a_{ji}$
$\boldsymbol{A} = \left(\boldsymbol{A}^T\right)^{-1}$	real orthogonal	$\sum_k a_{ik} a_{jk} = \sum_k a_{ki} a_{kj} = \delta_{ij}$
$\boldsymbol{A} = \boldsymbol{A}^*$	real matrix	$a_{ij} = a_{ij}^*$
$\boldsymbol{A} = \boldsymbol{A}^\dagger$	hermitian	$a_{ij} = a_{ji}^*$
$\boldsymbol{A} = \left(\boldsymbol{A}^\dagger\right)^{-1}$	unitary	$\sum_k a_{ik} a_{jk}^* = \sum_k a_{ki}^* a_{kj} = \delta_{ij}$

Some famous Matrices

- Diagonal if $a_{ij} = 0$ for $i \neq j$
- Upper triangular if $a_{ij} = 0$ for $i > j$
- Lower triangular if $a_{ij} = 0$ for $i < j$
- Upper Hessenberg if $a_{ij} = 0$ for $i > j + 1$
- Lower Hessenberg if $a_{ij} = 0$ for $i < j + 1$
- Tridiagonal if $a_{ij} = 0$ for $|i - j| > 1$
- Lower banded with bandwidth p: $a_{ij} = 0$ for $i > j + p$
- Upper banded with bandwidth p: $a_{ij} = 0$ for $i < j + p$
- Banded, block upper triangular, block lower triangular....

Matrix Features

Some equivalent statements for square matrices

For an $n \times n$ matrix $\boldsymbol{A}$ the following properties are all equivalent

- If the inverse of $\boldsymbol{A}$ exists, $\boldsymbol{A}$ is nonsingular.
- The equation $\boldsymbol{Ax} = 0$ implies $\boldsymbol{x} = 0$.
- The rows of $\boldsymbol{A}$ form a basis of $\mathbb{C}^n$.
- The columns of $\boldsymbol{A}$ form a basis of $\mathbb{C}^n$.
- $\boldsymbol{A}$ is a product of elementary matrices. Can you name one example?
- 0 is not an eigenvalue of $\boldsymbol{A}$.

Important Mathematical Operations

The basic matrix operations that we will deal with are addition and subtraction

$$\boldsymbol{A} = \boldsymbol{B} \pm \boldsymbol{C} \Longrightarrow a_{ij} = b_{ij} \pm c_{ij},$$

and scalar-matrix multiplication

$$\boldsymbol{A} = \gamma \boldsymbol{B} \Longrightarrow a_{ij} = \gamma b_{ij}.$$

Vector-matrix and Matrix-matrix multiplication

We have also vector-matrix multiplications

$$\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x} \Longrightarrow y_i = \sum_{j=0}^{n-1} a_{ij} x_j,$$

and matrix-matrix multiplications

$$\boldsymbol{A} = \boldsymbol{B}\boldsymbol{C} \Longrightarrow a_{ij} = \sum_{k=0}^{n-1} b_{ik} c_{kj},$$

and transpositions of a matrix

$$\boldsymbol{A} = \boldsymbol{B}^T \Longrightarrow a_{ij} = b_{ji}.$$

Important Mathematical Operations

Similarly, important vector operations that we will deal with are addition and subtraction

$$\boldsymbol{x} = \boldsymbol{y} \pm \boldsymbol{z} \Longrightarrow x_i = y_i \pm z_i,$$

scalar-vector multiplication

$$\boldsymbol{x} = \gamma \boldsymbol{y} \Longrightarrow x_i = \gamma y_i,$$

Other important mathematical operations

and vector-vector multiplication (called Hadamard multiplication)

$$\boldsymbol{x} = \boldsymbol{yz} \Longrightarrow x_i = y_i z_i.$$

Finally, as already metnioned, the inner or so-called dot product resulting in a constant

$$x = \boldsymbol{y}^T \boldsymbol{z} \Longrightarrow x = \sum_{j=0}^{n-1} y_j z_j,$$

and the outer product, which yields a matrix,

$$\boldsymbol{A} = \boldsymbol{yz}^T \Longrightarrow a_{ij} = y_i z_j,$$

Defining basis states and quantum mechanical operators

We extend now to quantum mechanics our definitions of vectors, matrices and more.
We start by defining a state vector $\boldsymbol{x}$ (meant to represent various quantum mechanical degrees of freedom) with n components as

$$\boldsymbol{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \dots \\ \dots \\ x_{n-1} \end{bmatrix}.$$

Dirac notation

Throughout these notes we will use the so-called Dirac **bra-ket** formalism and we will replace the above standard boldfaced notation for a vector with

$$\boldsymbol{x} = |x\rangle = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \dots \\ \dots \\ x_{n-1} \end{bmatrix},$$

and

$$\boldsymbol{x}^\dagger = \langle x| = \begin{bmatrix} x_0^* & x_1^* & x_2^* & \dots & \dots & x_{n-1}^* \end{bmatrix},$$

Inner product in Dirac notation

With a given vector $|x\rangle$, we define the inner product as

$$\langle x|x\rangle = \sum_{i=0}^{n-1} x_i^* x_i = |x_0|^2 + |x_1|^2 + \cdots + |x_{n-1}|^2.$$

For two arbitrary vectors $|x\rangle$ and $|y\rangle$ with the same lentgh, we have the general expression

$$\langle y|x\rangle = \sum_{i=0}^{n-1} y_i^* x_i = y_0^* x_0 + y_1^* x_1 + \cdots + y_{n-1}^* x_{n-1}.$$

The inner product is a real number

Note well that the inner product $\langle x|x\rangle$ is always a real number while for a two different vectors $\langle y|x\rangle$ is in general not equal to $\langle x|y\rangle$, as can be seen from the example in the next slide.

We note in bypassing that $|x\rangle^\dagger = \langle x|$, $\langle x|^\dagger = |x\rangle$ and $(|x\rangle^\dagger)^\dagger = |x\rangle$.

Examples

Let us assume that $|x\rangle$ is given by

$$|x\rangle = \begin{bmatrix} 1-i \\ 2+i \end{bmatrix}.$$

The inner product gives us

$$\langle x|x\rangle = (1+i)(1-i) + (2-i)(2+i) = 7,$$

a real number.

Norm

We can use the norm/inner product to normalize the vector $|x\rangle$ and obtain

$$|x\rangle = \frac{1}{\sqrt{7}} \begin{bmatrix} 1-\imath \\ 2+\imath \end{bmatrix}.$$

As another example, consider the two vectors

$$|x\rangle = \begin{bmatrix} -1 \\ 2\imath \\ 1 \end{bmatrix},$$

and

$$|y\rangle = \begin{bmatrix} 1 \\ 0\imath \\ \imath \end{bmatrix}.$$

We see that the inner products $\langle x|y\rangle = -1+\imath$, which is not the same as $\langle y|x\rangle = -1-\imath$. This leads to the important rule

$$\langle x|y\rangle^* = \langle y|x\rangle.$$

Outer products

In addition to inner products between vectors/states, the outer product plays a central role in all of quantum mechanics. It is defined as

$$|x\rangle\langle y| = \begin{bmatrix} x_0 y_0^* & x_0 y_1^* & x_0 y_2^* & \cdots & \cdots & x_0 y_{n-2}^* & x_0 y_{n-1}^* \\ x_1 y_0^* & x_1 y_1^* & x_1 y_2^* & \cdots & \cdots & x_1 y_{n-2}^* & x_1 y_{n-1}^* \\ x_2 y_0^* & x_2 y_1^* & x_2 y_2^* & \cdots & \cdots & x_2 y_{n-2}^* & x_2 y_{n-1}^* \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ x_{n-2} y_0^* & x_{n-2} y_1^* & x_{n-2} y_2^* & \cdots & \cdots & x_{n-2} y_{n-2}^* & x_{n-2} y_{n-1}^* \\ x_{n-1} y_0^* & x_{n-1} y_1^* & x_{n-1} y_2^* & \cdots & \cdots & x_{n-1} y_{n-2}^* & x_{n-1} y_{n-1}^* \end{bmatrix}$$

Different operators and gates

In quantum computing, the so-called Pauli matrices, and other simple 2×2 matrices, play an important role, ranging from the setup of quantum gates to a rewrite of creation and annihilation operators and other quantum mechanical operators. Let us start with the familiar Pauli matrices and remind ourselves of some of their basic properties.

The Pauli matrices are defined as

$$\sigma_x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$

$$\sigma_y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix},$$

and

$$\sigma_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

Properties of Pauli matrices

It is easy to show that the matrices obey the properties (being involutory)

$$\sigma_x\sigma_x = \sigma_y\sigma_y = \sigma_z\sigma_z = I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

that is their products with themselves result in the identity matrix $\boldsymbol{I}$. Furthermore, the Pauli matrices are unitary matrices meaning that their inverses are equal to their hermitian conjugated matrices. The determinants of the Pauli matrices are all equal to -1, as can be easily verified.

Commutation relations

The Pauli matrices obey also the following commutation rules

$$[\sigma_x, \sigma_y] = 2\imath\sigma_z.$$

Before we proceed with other matrices and how they can be used to operate on various quantum mechanical states, let us try to define various basis sets and their pertinent notations. We will often refer to these basis states as our computational basis.

Definition of Computational basis states

Assume we have a two-level system where the two states are represented by the state vectors $|\phi_0\rangle$ and $|\phi_1\rangle$, respectively. These states could represent selected or effective degrees of freedom for either a single particle (fermion or boson) or they could represent effective many-body degrees of freedon. In actual realizations of quantum computing we search often for candidate systems where we can use some low-lying states as computational basis states. But we are not limited to quantum computing. When doing many-body physics, due to the exploding degrees of freedom, we normally search after effective ways by which we can reduce the involved dimensionalities to a number of degrees of freedom we can handle by a given many-body method.

Projection operators

We will now relabel the above two states as two orthogonal and normalized basis (ONB) states

$$|\phi_0\rangle = |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

and

$$|\phi_1\rangle = |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Identity and projection operators

It is straight forward to see that $\langle 1|0\rangle = 0$. With these two states we can define the define the identity operator $\boldsymbol{I}$ as the sum of the outer products of these two states, namely

$$\boldsymbol{I} = \sum_{i=0}^{i=1} |i\rangle\langle i| = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

We can further define the projection operators

$$\boldsymbol{P} = |0\rangle\langle 0| = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix},$$

and

$$\boldsymbol{Q} = |1\rangle\langle 1| = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

Idempotent operators

We note that $P^2 = P$, $Q^2 = Q$ (the operators are idempotent) and that their determinants are zero, meaning in turn that we cannot use these operators for unitary/orthogonal transformations. However, they play important roles in defining effective Hilbert spaces for many-body studies. Finally, before proceeding we note also that the two matrices commute and we have $\boldsymbol{PQ} = 0$ and $[\boldsymbol{P}, \boldsymbol{Q}] = 0$.

Superposition and more

Using the properties of ONBs we can expand a new state in terms of the above states. These states could also form a basis which is an eigenbasis of a selected Hamiltonian (more of this below).
We define now a new state which is a linear expansion in terms of these computational basis states

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

where the coefficients $\alpha = \langle 0|\psi\rangle$ and $\beta = \langle 1|\psi\rangle$ reresent the overlaps between the computational basis states and the state $|\psi\rangle$.
In quantum speech, we say the state is in a superposition of the states $|0\rangle$ and $|1\rangle$.

Inner products

Computing the inner product of $|\psi\rangle$ we obtain

$$\langle\psi|\psi\rangle = |\alpha|^2\langle 0|0\rangle + |\beta|^2\langle 1|1\rangle = |\alpha|^2 + |\beta|^2 = 1,$$

since the new basis, which is defined in terms of a a unitary/orthogonal transformation, preserves the orthogonality and norm of the original computational basis $|0\rangle$ and $|1\rangle$. To see this, consider the unitary transformation (show derivation of preserving orthogonality).

Acting with projection operators

If we now act with the projection operators $\boldsymbol{P}$ and $\boldsymbol{Q}$ on the state $|\psi\rangle$ we get

$$\boldsymbol{P}|\psi\rangle = |0\rangle\langle 0|(\alpha|0\rangle + \beta|1\rangle) = \alpha|0\rangle,$$

that is we **project** out the $|0\rangle$ component of the state $|\psi\rangle$ with the coefficient α while $\boldsymbol{Q}$ projects out the $|1\rangle$ component with coefficient β as seen from

$$\boldsymbol{Q}|\psi\rangle = |1\rangle\langle 1|(\alpha|0\rangle + \beta|1\rangle) = \beta|1\rangle.$$

The above results can easily be derived by multiplying the pertinent matrices with the vectors $|0\rangle$ and $|1\rangle$, respectively.

Density matrix

Using the above linear expansion we can now define the density matrix of the state $|\psi\rangle$ as the outer product

$$\boldsymbol{\rho} = |\psi\rangle\langle\psi| = \alpha\alpha^*|0\rangle\langle 0| + \alpha\beta^*|0\rangle\langle 1| + \beta\alpha^*|1\rangle\langle 0| + \beta\beta^*|1\rangle\langle 1|,$$

which leads to

$$\boldsymbol{\rho} = \begin{bmatrix} \alpha\alpha^* & \alpha\beta^* \\ \beta\alpha^* & \beta\beta^* \end{bmatrix}.$$

Finally, we note that the trace of the density matrix is simply given by unity

$$\mathrm{tr}\boldsymbol{\rho} = \alpha\alpha^* + \beta\beta^* = 1.$$

Other important matrices

We present other operators (as matrices) which play an important role in quantum computing, the so-called Hadamard matrix (or gate as we will use later)

$$\boldsymbol{H} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

The action of the operator $\boldsymbol{H}$ on a computational basis state like $|0\rangle$ gives

$$\boldsymbol{H}|0\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle),$$

and

$$\boldsymbol{H}|1\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle),$$

that is we create a superposition of the states $|0\rangle$ and $|1\rangle$.

Phase matrix

Another famous operation is the phase matrix given by

$$\boldsymbol{S} = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}.$$

Tensor products

Consider now two vectors with length $n = 2$, with elements

$$|x\rangle = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix},$$

and

$$|y\rangle = \begin{bmatrix} y_0 \\ y_1 \end{bmatrix}.$$

The tensor product of these two vectors is defined as

$$|x\rangle \otimes |y\rangle = |xy\rangle = \begin{bmatrix} x_0 y_0 \\ x_0 y_1 \\ x_1 y_0 \\ x_1 y_1 \end{bmatrix},$$

which is now a vector of length 4.

Examples of tensor products

If we now go back to our original one-qubit basis states, we can form teh following tensor products

$$|0\rangle \otimes |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = |00\rangle,$$

$$|0\rangle \otimes |1\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = |01\rangle.$$

More states

$$|1\rangle \otimes |0\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = |10\rangle,$$

and finally

$$|1\rangle \otimes |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = |11\rangle.$$

Three qubits

We have now four different states and we could try to make a new list by relabeling the states as follows $|00\rangle = |0\rangle$, $|01\rangle = |1\rangle$, $|10\rangle = |2\rangle$, $|11\rangle = |3\rangle$.
In similar ways we can define the tensor product of three qubits (or single-particle states) as

$$|0\rangle \otimes |0\rangle \otimes |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = |000\rangle,$$

which is a new vector of length eight.

Generalizing

We note that with a single-particle basis given the states $|0\rangle$ and $|1\rangle$ we can, with N particles construct 2^N different states. This is something we can generalize to

- discuss ways of labeling states
- how to write a code which does it

Tensor products of matrices

The tensor product of two 2×2 matrices $\boldsymbol{A}$ and $\boldsymbol{B}$ is given by

$$\boldsymbol{A} \times \boldsymbol{B} = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix} \otimes \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix} = \begin{bmatrix} a_{00}b_{00} & a_{00}b_{01} & a_{01}b_{00} & a_{01}b_{01} \\ a_{00}b_{10} & a_{00}b_{11} & a_{01}b_{10} & a_{01}b_{11} \\ a_{10}b_{00} & a_{10}b_{01} & a_{11}b_{00} & a_{11}b_{01} \\ a_{10}b_{10} & a_{10}b_{11} & a_{11}b_{10} & a_{11}b_{11} \end{bmatrix}$$

Measurements

The probability of a measurement on a quantum system giving a certain result is determined by the weight of the relevant basis state in the state vector. After the measurement, the system is in a state that corresponds to the result of the measurement. The operators and gates discussed below are examples of operations we can perform on specific states.
We consider the state

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Definitions of measurements

1. A measurement can yield only one of the above states, either $|0\rangle$ or $|1\rangle$.
2. The probability of a measurement resulting in $|0\rangle$ is $\alpha^*\alpha = |\alpha|^2$.
3. The probability of a measurement resulting in $|1\rangle$ is $\beta^*\beta = |\beta|^2$.
4. And we note that the sum of the outcomes gives $\alpha^*\alpha + \beta^*\beta = 1$ since the two states are normalized.

After the measurement, the state of the system is the state associated with the result of the measurement.
We have already encountered the projection operators P and Q.
Let us now look at other types of operations we can make on qubit states.

Different operators and gates

In quantum computing, the so-called Pauli matrices, and other simple 2×2 matrices, play an important role, ranging from the setup of quantum gates to a rewrite of creation and annihilation operators and other quantum mechanical operators. Let us start with the familiar Pauli matrices and remind ourselves of some of their basic properties.
Assume we operate with σ_x on our basis state $|0\rangle$. This gives

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

that is we switch from $|0\rangle$ to $|1\rangle$ (often called a spin flip operation) and similary we have

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

More on Pauli matrices

This matrix plays an important role in quantum computing since we can link this with the classical **NOT** operation. If we send in bit 0, the **NOT** gate outputs bit 1 and vice versa. We can use the σ_x matrix to implement the quantum mechanical equivalent of a classical **NOT** gate. If we input what we could represent as bit 0 in terms of the basis state $|0\rangle$, operating on this state results in the state $|1\rangle$, which we in turn can interpret as the classical bit 1.

Linear superposition

If we have a linear superposition of these states we obtain

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \beta \\ \alpha \end{bmatrix}.$$

The σ_y matrix introduces an imaginary sign, which we will later encounter in terms of so-called phase-shift operations.

The σ_z matrix

The σ_z matrix has the following effect

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

and

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix},$$

which we will also link with a specific phase-shift.

Unitarity

The matrices we introduced here are so-called unitary matrices. This is an important element in quantum mechanics since the evolution of a closed quantum system is described by operations involving unitary operations only.
We have defined a new state $|\psi_p\rangle$ as a linear expansion in terms of an orthogonal and normalized basis (our computational basis) ϕ_λ

$$|\psi_i\rangle = \sum_j u_{ij}|\phi_j\rangle. \tag{6}$$

Hamiltonians and basis functions

It is normal to choose a basis defined as the eigenfunctions of parts of the full Hamiltonian. The typical situation consists of the solutions of the one-body part of the Hamiltonian, that is we have

$$\hat{h}_0|\phi_i\rangle = \epsilon_i|\phi_i\rangle.$$

This is normally referred to as a single-particle basis $|\phi_i(\mathrm{r})\rangle$, defined by the quantum numbers i and r.

Unitary transformations

A unitary transformation is important since it keeps the orthogonality. To see this consider first a basis of vectors $\mathbf{v}_i$,

$$\mathbf{v}_i = \begin{bmatrix} v_{i1} \\ \cdots \\ \cdots \\ v_{in} \end{bmatrix}$$

We assume that the basis is orthogonal, that is

$$\mathbf{v}_j^T \mathbf{v}_i = \delta_{ij}.$$

An orthogonal or unitary transformation

$$\mathbf{w}_i = \mathbf{U}\mathbf{v}_i,$$

preserves the dot product and orthogonality since

$$\mathbf{w}_j^T \mathbf{w}_i = (\mathbf{U}\mathbf{v}_j)^T \mathbf{U}\mathbf{v}_i = \mathbf{v}_j^T \mathbf{U}^T \mathbf{U}\mathbf{v}_i = \mathbf{v}_j^T \mathbf{v}_i = \delta_{ij}.$$

Orthogonality preserved

This means that if the coefficients $u_{p\lambda}$ belong to a unitary or orthogonal transformation (using the Dirac bra-ket notation)

$$|\psi_i\rangle = \sum_j u_{ij}|\phi_j\rangle.$$

orthogonality is preserved.
This propertry is extremely useful when we build up a basis of many-body determinant based states.
Note also that although a basis $\{|\phi_i\rangle\}$ contains an infinity of states, for practical calculations we have always to make some truncations.

Example

Assume we have two one-qubit states represented by

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix},$$

and

$$|\phi\rangle = \gamma|0\rangle + \delta|1\rangle = \begin{bmatrix} \gamma \\ \delta \end{bmatrix}.$$

We assume that the state $|\phi\rangle$ is obtained through a unitary transformation of $|\psi\rangle$ through a matrix $\boldsymbol{U}$ with its hermitian conjugate $\boldsymbol{U}^\dagger$ with matrix elements $u_{ij}^\dagger = u_{ji}^*$ and $\boldsymbol{I} = \boldsymbol{U}\boldsymbol{U}^\dagger = \boldsymbol{U}^\dagger\boldsymbol{U}$.

Inverse of unitary matrices

Note that this means that the hermitian conjugate of a unitary matrix is equal to its inverse. This has important consequences for what is called reversibility. We say quantum mechanics is a theory which is reversible with a probabilistic determinism. Classical mechanics on the other is reversible in a deterministic way, that is, knowing all initial conditions we can in principle determine the future motion of an object which obey the laws of motion of classical mechanics.
We have then

$$\begin{bmatrix} \gamma \\ \delta \end{bmatrix} = \begin{bmatrix} u_{00} & u_{01} \\ u_{10} & u_{11} \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix}.$$

New basis is also orthogonal

Since our original basis $|\psi\rangle$ is orthogonal and normalized with $|\alpha|^2 + |\beta|^2 = 1$, the new basis is also orthogonal and normalized, as we can see below here.
Since the inverse of a hermitian matrix is equal to its hermitian conjugate/adjoint), unitary transformations are always reversible.
Why are only unitary transformations allowed? The key lies in the way the inner product tranforms.
To see this we rewrite the new basis from the previous example in its two components as

$$|\phi\rangle_i = \sum_j u_{ij} |\psi\rangle_j,$$

or in terms of a matrix-vector notatio we have

$$|\phi\rangle = \boldsymbol{U}|\psi\rangle,$$

More on orthogonality

We have already assumed that $\langle\psi|\psi\rangle = |\alpha|^2 + |\beta|^2 = 1$.
We have that

$$\langle\phi|_i = \sum_j u_{ij}^* \langle\psi|_j,$$

or in terms of a matrix-vector notation we have

$$\langle\phi| = \langle\psi|\boldsymbol{U}^\dagger.$$

Note that the two vectors are row vectors now.
If we stay with this notation we have

$$\langle\phi|\phi\rangle = \langle\psi\boldsymbol{U}^\dagger\boldsymbol{U}|\psi\rangle = \langle\psi|\psi\rangle = 1!$$

Unitary transformations are rotations in state space which preserve the length (the square root of the inner product) of the state vector.

Entanglement

In order to study entanglement and why it is so important for quantum computing, we need to introduce some basic measures and useful quantities. These quantities are the spectral decomposition of hermitian operators, how these are then used to define measurements and how we can define so-called density operators (matrices). These are all quantities which will become very useful when we discuss entanglement and in particular how to quantify it. In order to define these quantities we need first to remind ourselves about some basic linear algebra properties of hermitian operators and matrices.

Basic properties of hermitian operators

The operators we typically encounter in quantum mechanical studies are

1. Hermitian (self-adjoint) meaning that for example the elements of a Hermitian matrix $\boldsymbol{U}$ obey $u_{ij} = u_{ji}^*$.
2. Unitary $\boldsymbol{U}\boldsymbol{U}^\dagger = \boldsymbol{U}^\dagger\boldsymbol{U} = \boldsymbol{I}$, where $\boldsymbol{I}$ is the unit matrix
3. The oparator $\boldsymbol{U}$ and its self-adjoint commute (often labeled as normal operators), that is $[\boldsymbol{U}, \boldsymbol{U}^\dagger] = 0$. An operator is **normal** if and only if it is diagonalizable. A Hermitian operator is normal.

Unitary operators in a Hilbert space preserve the norm and orthogonality. If $\boldsymbol{U}$ is a unitary operator acting on a state $|\psi_j\rangle$, the action of

$$|\phi_i\rangle = \boldsymbol{U}|\psi_j\rangle,$$

preserves both the norm and orthogonality, that is $\langle\phi_i|\phi_j\rangle = \langle\psi_i|\psi_j\rangle = \delta_{ij}$, as discussed earlier.

The Pauli matrices again

As example, consider the Pauli matrix σ_x. We have already seen that this matrix is a unitary matrix. Consider then an orthogonal and normalized basis $|0\rangle^\dagger = \begin{bmatrix}1 & 0\end{bmatrix}$ and $|1\rangle^\dagger = \begin{bmatrix}0 & 1\end{bmatrix}$ and a state which is a linear superposition of these two basis states

$$|\psi_a\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle.$$

A new state $|\psi_b\rangle$ is given by

$$|\psi_b\rangle = \sigma_x|\psi_a\rangle = \alpha_0|1\rangle + \alpha_1|0\rangle.$$

Spectral Decomposition

An important technicality which we will use in the discussion of density matrices, entanglement, quantum entropies and other properties is the so-called spectral decomposition of an operator. Let $|\psi\rangle$ be a vector in a Hilbert space of dimension n and a hermitian operator $\boldsymbol{A}$ defined in this space. Assume $|\psi\rangle$ is an eigenvector of $\boldsymbol{A}$ with eigenvalue λ, that is

$$\boldsymbol{A}|\psi\rangle = \lambda|\psi\rangle = \lambda \boldsymbol{I}|\psi\rangle,$$

where we used $\boldsymbol{I}|\psi\rangle = 1|\psi\rangle$. Subtracting the right hand side from the left hand side gives

$$[\boldsymbol{A} - \lambda \boldsymbol{I}]\,|\psi\rangle = 0,$$

which has a nontrivial solution only if the determinant $\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = 0$.

ONB again and again

We define now an orthonormal basis $|i\rangle = \{|0\rangle, |1\rangle, \ldots, |n-1\rangle$ in the same Hilbert space. We will assume that this basis is an eigenbasis of $\boldsymbol{A}$ with eigenvalues λ_i
We expand a new vector using this eigenbasis of $\boldsymbol{A}$

$$|\psi\rangle = \sum_{i=0}^{n-1} \alpha_i |i\rangle,$$

with the normalization condition $\sum_{i=0}^{n-1} |\alpha_i|^2$. Acting with $\boldsymbol{A}$ on this new state results in

$$\boldsymbol{A}|\psi\rangle = \sum_{i=0}^{n-1} \alpha_i \boldsymbol{A}|i\rangle = \sum_{i=0}^{n-1} \alpha_i \lambda_i |i\rangle.$$

Projection operators

If we then use that the outer product of any state with itself defines a projection operator we have the projection operators

$$\boldsymbol{P}_{\psi} = |\psi\rangle\langle\psi|,$$

and

$$\boldsymbol{P}_{j} = |j\rangle\langle j|,$$

we have that

$$\boldsymbol{P}_{j}|\psi\rangle = |j\rangle\langle j| \sum_{i=0}^{n-1} \alpha_i |i\rangle = \sum_{i=0}^{n-1} \alpha_i |j\rangle\langle j|i\rangle.$$

Further manipulations

This results in

$$\boldsymbol{P}_j|\psi\rangle = \alpha_j|j\rangle,$$

since $\langle j|i\rangle$. With the last equation we can rewrite

$$\boldsymbol{A}|\psi\rangle = \sum_{i=0}^{n-1} \alpha_i \lambda_i |i\rangle = \sum_{i=0}^{n-1} \lambda_i \boldsymbol{P}_i|\psi\rangle,$$

from which we conclude that

$$\boldsymbol{A} = \sum_{i=0}^{n-1} \lambda_i \boldsymbol{P}_i.$$

Spectral decomposition

This is the spectral decomposition of a hermitian and normal operator. It is true for any state and it is independent of the basis. The spectral decomposition can in turn be used to exhaustively specify a measurement, as we will see in the next section.
As an example, consider two states $|\psi_a\rangle$ and $|\psi_b\rangle$ that are eigenstates of $\boldsymbol{A}$ with eigenvalues λ_a and λ_b, respectively. In the diagonalization process we have obtained the coefficients α_0, α_1, β_0 and β_1 using an expansion in terms of the orthogonal basis $|0\rangle$ and $|1\rangle$.

Explicit results

We have then

$$|\psi_a\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle,$$

and

$$|\psi_b\rangle = \beta_0|0\rangle + \beta_1|1\rangle,$$

with corresponding projection operators

$$\boldsymbol{P}_a = |\psi_a\rangle\langle\psi_a| = \begin{bmatrix} |\alpha_0|^2 & \alpha_0\alpha_1^* \\ \alpha_1\alpha_0^* & |\alpha_1|^* \end{bmatrix},$$

and

$$\boldsymbol{P}_b = |\psi_b\rangle\langle\psi_b| = \begin{bmatrix} |\beta_0|^2 & \beta_0\beta_1^* \\ \beta_1\beta_0^* & |\beta_1|^* \end{bmatrix}.$$

The spectral decomposition

The results from the previous slide gives us the following spectral decomposition of $\boldsymbol{A}$

$$\boldsymbol{A} = \lambda_a |\psi_a\rangle\langle\psi_a| + \lambda_b |\psi_b\rangle\langle\psi_b|,$$

which written out in all its details reads

$$\boldsymbol{A} = \lambda_a \begin{bmatrix} |\alpha_0|^2 & \alpha_0\alpha_1^* \\ \alpha_1\alpha_0^* & |\alpha_1|^* \end{bmatrix} + \lambda_b \begin{bmatrix} |\beta_0|^2 & \beta_0\beta_1^* \\ \beta_1\beta_0^* & |\beta_1|^* \end{bmatrix}.$$

Exercise set for first week, to be discussed January 28

The exercises here are mostly meant as a reminder of basic linear algebra properties. We have also included some exercises which prepare for the first project. The first project deals with implementing the so-called **Variational Quantum Eigensolver** algorithm for finding the eigenvalues and eigenvectors of selected Hamiltonians.

1: Commutator identities

Prove the following commutator relations for different operators (marked with a hat)

1. $[\hat{A} + \hat{B}, \hat{C}] = [\hat{A}, \hat{C}] + [\hat{B}, \hat{C}]$;
2. $[\hat{A}, \hat{B}\hat{C}] = [\hat{A}, \hat{B}]\hat{C} + \hat{B}[\hat{A}, \hat{C}]$; and
3. $[\hat{A}, [\hat{B}\hat{C}]] = [\hat{B}, [\hat{C}, \hat{A}]] + [\hat{C}, [\hat{A}, \hat{B}]] = 0$ (the so-called Jacobi identity).

2: Pauli matrices

1. Set up the commutation rules for the Pauli matrices, that is find $[\sigma_i, \sigma_j]$ where $i, j = x, y, z$.
2. We define $\boldsymbol{X} = \sigma_x$, $\boldsymbol{Y} = \sigma_y$ and $\boldsymbol{Z} = \sigma_z$. Show that $\boldsymbol{XX} = \boldsymbol{YY} = \boldsymbol{ZZ} = \boldsymbol{I}$.
3. Which one of the Pauli matrices has the qubit basis $|0\rangle$ and $|1\rangle$ as eigenbasis? What are the eigenvalues?

3: Shared eigenvectors

Prove that if two operators $\hat{A}$ and $\hat{B}$ commute they will share a basis of eigenstates

4: One-qubit basis and Pauli matrices

Write a function (in Python for example) which sets up a one-qubit basis and apply the various Pauli matrices to these basis states.

5: Hadamard and Phase gates

Apply the Hadamard and Phase matrices (or gates) to the same one-qubit basis states and study their actions on these states. Write a code which applies these matrices to the same one-qubit basis.